OurTearoa

INF803 - Cloud Computing Project
Cloud Web Application Development and Deployment

Submission Date: 16 October 2026

Mathias Jørgensen
Student ID: 26344066
Auckland University of Technology
cvm6264@autuni.ac.nz

Tim Schiller
Student ID: 26347002
Auckland University of Technology
zpn6423@autuni.ac.nz



OurTearoa

SHARE YOUR JOURNEY

Preamble

Project Deliverables & Access Links

Presentation Video	Presentation Video (Public URL, < 5 minutes)
GitHub Repository	https://github.com/bundgaard1/OurTearoa
Live Web Client URL	https://ourtearoa.duckdns.org
Live REST API Base	https://ourtearoa.duckdns.org/api

Member Contributions

Member Name	Student ID	Assigned Components & Cloud Tasks
Mathias Jorgensen	26344066	Setup AWS (S3 storage, EC2 compute, RDS PostgreSQL), Project Owner, Backend API Development (Express/Prisma), S3 asset integration
Tim Schiller	26347002	Frontend SPA Development (Angular 22, Signals), Project Manager, JWT Auth Architecture, Nginx/ Certbot TLS setup, GitHub Actions CI/CD Pipeline

1. Introduction

1.1. Project Overview and Domain

OurTearoa is a cloud-native travel exploration platform engineered to streamline how users discover and experience New Zealand (Aotearoa). The platform enables visitors and travellers to browse regional destinations, explore granular location details, and submit authenticated ratings and reviews. Registered users can curate sequential, date-ordered travel itineraries across multiple destinations.

Architecturally, the application enforces a strict separation of concerns through a decoupled three-tier model. A standalone Angular single-page application functions purely as the presentation tier, consuming stateless JSON endpoints exposed by an Express 5 REST API. All communication between the frontend client and backend services is secured over TLS/HTTPS, ensuring data transit integrity while allowing the API tier to be tested, scaled, and managed independently of the client interface.

1.2. Technology Stack

- **Client Tier:** Built with Angular 22 as a Single Page Application (SPA), leveraging standalone components, reactive signals for state management, Angular Router, and Vitest for unit testing.
- **Application Tier:** Implemented using Express 5 and TypeScript running directly on Node.js 26 via tsx, managed in production with PM2 process persistence under systemd
- **Data Tier:** Managed PostgreSQL 16 on AWS RDS (ap-southeast-2), accessed via Prisma 7 ORM utilizing the @prisma/adapter-pg driver over verified TLS connections.
- **Infrastructure & Network:** Compute hosted on an AWS EC2 t3.micro instance running Amazon Linux 2023, static media offloaded to AWS S3, reverse proxy and TLS termination handled by Nginx with Let's Encrypt certificates, and dynamic DNS mapped via DuckDNS (ourtearoa.duckdns.org).
- **Development:** For CI/CD we used Github workflows. And

2. Background and Requirements Analysis

2.1. Functional Requirements

> **What to write**
> - Summarize the key user story groupings from the design document:
>   - Visitor requirements (US-01 to US-04: catalogue browsing, regional filtering, place details, reading reviews).
>   - Registered Traveller requirements (US-05 to US-13: JWT authentication, favourites management, star reviews, date-ordered trip itineraries).
>   - Contributor & Administrative features (US-14, US-15: place creation and media upload).

2.2. Cloud Engineering & Evaluator Requirements

What to write

- Marker/Evaluator accessibility (US-17, US-18): documented REST endpoints testable without the UI, and a dedicated `/api/health` check reporting DB status.
- Non-functional cloud requirements: HTTPS encryption on a custom DNS domain (US-20), stateless server architecture, and resilience on minimal compute tiers.

3. Project Management

3.1. Work Breakdown Structure (WBS)

What to write & Artifact

- Break down development across sprints: AWS infrastructure setup, Database migrations, Backend REST controllers, Angular client views, CI/CD pipeline automation, and smoke testing.
- Insert WBS visual diagram.

Figure 3.1: Work Breakdown Structure (WBS)
(Insert screenshot image file here)

3.2. 3.2 Schedule, Milestones & Sprint Progress

What to write & Artifact

- Show milestone progression from project kickoff, API smoke test validation, to production deployment.

Figure 3.2: Project Timeline & Milestone Delivery Schedule
(Insert screenshot image file here)

3.3. 3.3 Meeting Minutes & Collaboration Logs

What to write

- Document regular coordination syncs between Mathias and Tim (dates, decisions, task assignments).

4. 4. Worklog and Collaboration Evidence

4.1. 4.1 Project Management Evidence (Trello / Kanban)

What to write & Screenshots

- Provide URL to your online board.
- Insert snapshots of task cards showing issue tracking, checklist completion, and timestamps.

Figure 4.1: Kanban Board with Backlog, Progress, and Production Done states

(Insert screenshot image file here)

Figure 4.2: Sample Issue Cards with Timestamped Activity Logs

(Insert screenshot image file here)

4.2. 4.2 Git Repository Contributions

What to write & Screenshots

- Provide evidence of commits from both Mathias and Tim showing balanced contributions across frontend, backend, and cloud config.
- Show GitHub Insights commit distribution and confirm no commit history rewriting was performed.

Figure 4.3: GitHub Commit Frequency and Contributor Distribution Graph

(Insert screenshot image file here)

Figure 4.4: GitHub Commit Log with Timestamp History

(Insert screenshot image file here)

5. High-Level and Detailed Design

5.1. Three-Tier Architecture

What to write & Diagram

- Present your 3-tier architecture: Browser SPA -> Nginx (Port 443) -> Express API (127.0.0.1:3000) -> AWS RDS PostgreSQL (Port 5432).
- Detail the step-by-step request lifecycle: TLS termination at Nginx, proxying /api/*, Prisma connection over TLS to RDS, and JSON response serialization.

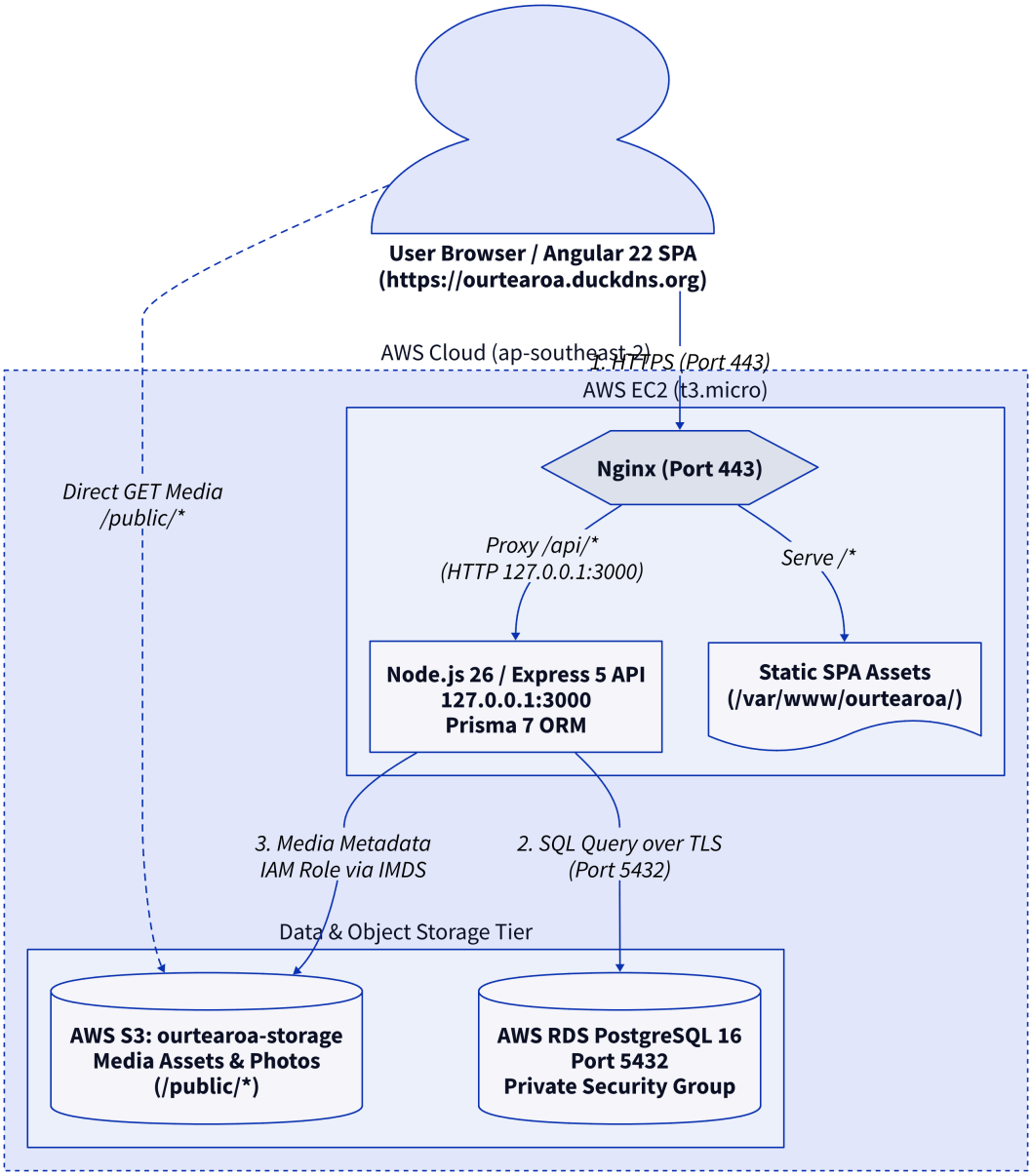


Figure 1: High-Level Three-Tier Cloud Architecture Diagram (EC2, Nginx, RDS, S3)

5.2. Request Lifecycle

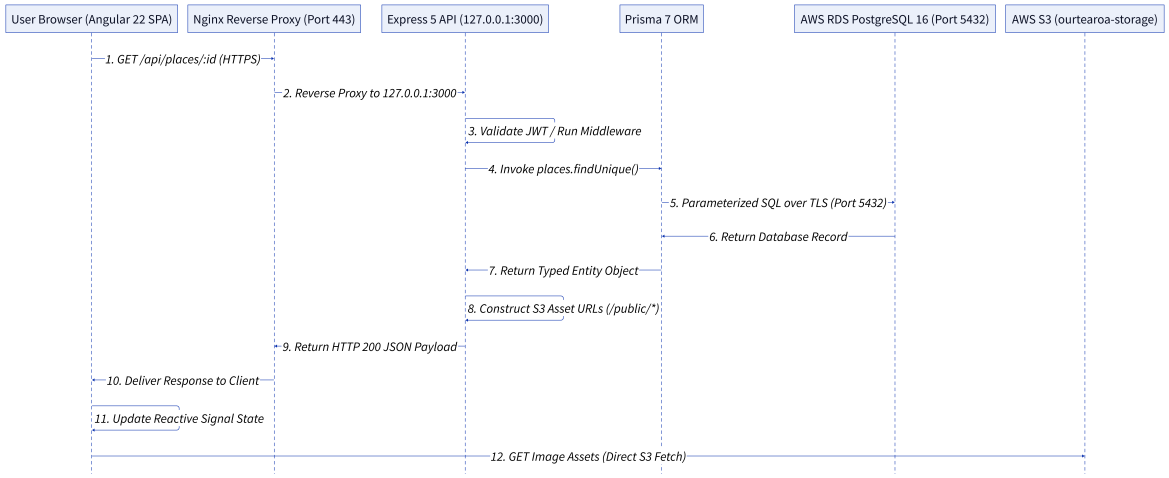


Figure 2:

5.3. Database Model & Relational Schema

What to write & Diagram

- Present the PostgreSQL relational schema managed by Prisma: User, Place, Review, Favorite, and ItineraryEntry models with relational cascade rules.

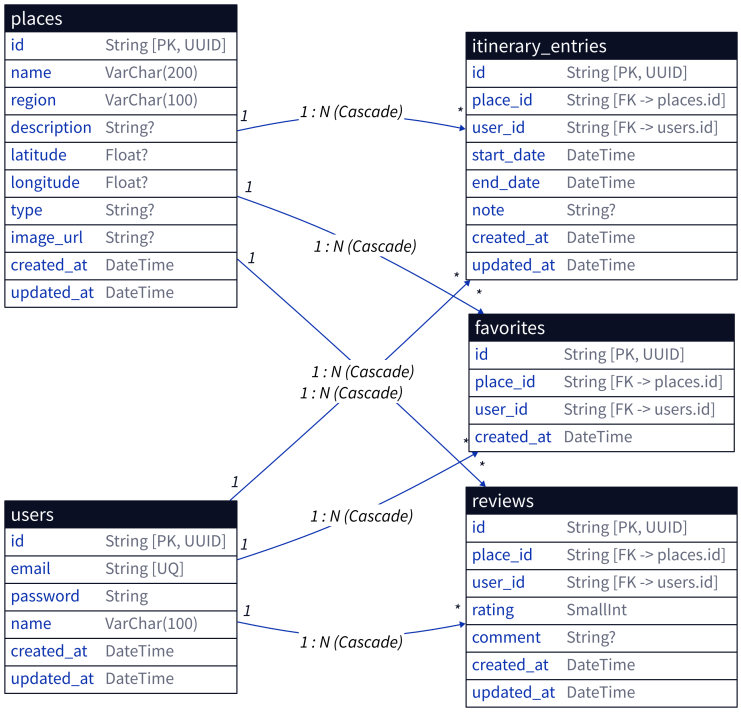


Figure 3: Hello World

5.4. Object Storage Architecture (AWS S3)

What to write

- Detail AWS S3 bucket configuration for place photography and user-uploaded media.
- Discuss AWS SDK integration (@aws-sdk/client-s3), public-read bucket policies, and CORS configuration.

6. 6. Details on Cloud Implementation

6.1. 6.1 Compute Provisioning & System Hardening (AWS EC2)

What to write & Screenshots

- Document AWS EC2 t3.micro instance setup with Amazon Linux 2023.
- Document the Linux swap file setup (critical for 1 GB RAM systems to avoid OOM crashes).
- Document Nginx web server configuration, Let's Encrypt Certbot setup, and DuckDNS dynamic DNS integration.
- Document PM2 process management running the TypeScript API (tsx) with systemd boot persistence.

Figure 6.1: EC2 Instance Configuration & Swap Space Verification

(Insert screenshot image file here)

Figure 6.2: PM2 Status Dashboard and Systemd Daemon Logs

(Insert screenshot image file here)

6.2. 6.2 Managed Cloud Database Setup (AWS RDS PostgreSQL)

What to write & Screenshots

- Document AWS RDS PostgreSQL 16 instance setup in ap-southeast-2 (confirming it is a managed service, not a local DB in VM).
- Show security group configuration: port 5432 restricted strictly to incoming traffic from the EC2 security group.
- Document SSL/TLS certificate verification using sslmode=verify-full and AWS global-bundle.pem.

Figure 6.3: AWS RDS Console - Database Status, Engine Version, and Subnet Groups
(Insert screenshot image file here)

Figure 6.4: RDS Security Group Inbound Rule Scoped to EC2 Security Group ID
(Insert screenshot image file here)

6.3. 6.3 Cloud Object Storage Provisioning (AWS S3)

What to write & Screenshots

- Show AWS S3 bucket dashboard, region selection, permissions, and media folder structure.

Figure 6.5: AWS S3 Bucket Configuration and Uploaded Place Asset Objects
(Insert screenshot image file here)

6.4. 6.4 CI/CD Pipeline & Zero-Downtime Deployment

What to write & Screenshots

- Document your `.github/workflows/deploy.yaml` GitHub Actions pipeline:
  1. Compiling Angular bundle on GitHub runner (offloading memory pressure from the EC2 instance).
  2. Rsyncing artifacts to `/var/www/ourtearoa/` and enforcing SELinux `httpd_sys_content_t` contexts.
  3. SSH execution: `git pull`, `prisma migrate deploy`, and PM2 API reloading.
- Show secrets isolation (GitHub Actions secrets storing SSH keys and database URIs).

Figure 6.6: GitHub Actions Workflow Run Executions Proving Automated Deployment

(Insert screenshot image file here)

7. 7. User Manual

7.1. 7.1 Accessing the Application

What to write

- Instructions for opening https://ourtearoa.duckdns.org in modern web browsers over HTTPS.

7.2. 7.2 End-User Workflow Walkthrough

What to write & Screenshots

- Step 1: Browsing Places and filtering destinations by region.
- Step 2: Account creation (/register) and authentication (/login).
- Step 3: Favouriting places and building a custom itinerary with start/end dates.
- Step 4: Submitting star ratings and comments on place detail views.

Figure 7.1: UI Walkthrough - Place Catalogue and Regional Filtering

(Insert screenshot image file here)

Figure 7.2: UI Walkthrough - Split-View Place Detail with Map and Reviews

(Insert screenshot image file here)

Figure 7.3: UI Walkthrough - Itinerary Builder Ordered by Date

(Insert screenshot image file here)

8. 8. RESTful API List

8.1. 8.1 API Overview & Evaluation Details

> **What to write**
> - Re-state marker base URL: https://ourtearoa.duckdns.org/api
> - Explain how markers can test endpoints using JWT Bearer headers obtained via /api/auth/login.
> - Detail the automated health endpoint: GET /api/health returning database connectivity status.

8.2. 8.2 Endpoints Specification Table

Method	Path	Auth	Status	Description
GET	/health	No	200	API and RDS PostgreSQL connectivity check
POST	/auth/register	No	201	Register user (email, name, password)
POST	/auth/login	No	200	Authenticate user and issue Bearer JWT
GET	/users/me	Bearer	200	Get authenticated user profile
GET	/users/:id	Bearer	200	Get user profile summary by ID
DELETE	/users/me	Bearer	204	Delete account and cascade related records
GET	/favorites	Bearer	200	Fetch user's saved favorite places
POST	/favorites	Bearer	201	Add place to favorites shortlist (placeId)
DELETE	/favorites/:id	Bearer	204	Remove place from favorites
GET	/itinerary	Bearer	200	Fetch itinerary entries ordered by date
POST	/itinerary	Bearer	201	Add itinerary entry (placeId, dates, note)
PUT	/itinerary/:id	Bearer	200	Update dates or notes on itinerary entry
DELETE	/itinerary/:id	Bearer	204	Remove entry from itinerary
GET	/places	No	200	List all places with regional filtering
GET	/places/:id	No	200	Detailed place information and coordinates
POST	/places	Bearer	201	Create new place catalogue entry
PUT	/places/:id	Bearer	200	Update existing place entry
DELETE	/places/:id	Bearer	204	Delete place from catalogue
GET	/places/:id/ reviews	No	200	Get user reviews for a specific place
POST	/reviews	Bearer	201	Create star rating and review (rating, comment)
PUT	/reviews/:id	Bearer	200	Update existing review
DELETE	/reviews/:id	Bearer	204	Remove own review

8.3. 8.3 Postman & Smoke Test Verification Evidence

What to write & Screenshots

- Show terminal output executing scripts/api-smoke-test.sh https://ourtearoa.duckdns.org/api.
- Include Postman screenshots showing request headers, live cloud URL, payload, and JSON responses for key POST and GET endpoints.

Figure 8.1: API Verification - GET /health returning status: UP, database: UP

(Insert screenshot image file here)

Figure 8.2: Postman Test - POST /auth/register and POST /auth/login issuing JWT

(Insert screenshot image file here)

Figure 8.3: Postman Test - POST /itinerary with JSON payload (201 Created)

(Insert screenshot image file here)

Figure 8.4: Automated Smoke Test Script Output (api-smoke-test.sh)

(Insert screenshot image file here)

9. 9. Summary

9.1. 9.1 Evaluation Against Brief

What to write

- Review project achievements against Option B requirements: fully decoupled 3-tier architecture, managed RDS persistence, public access, and automated CI/CD.

9.2. 9.2 Critical Engineering Reflections & Lessons Learned

What to write (Criterion 4 & Criterion 6)

- Memory Optimization: explain why running an Angular build on a t3.micro instance caused memory crashes, and how offloading the build step to GitHub Actions runners with a swap file fallback solved the problem.
- Cloud Security: discuss configuring TLS termination via Nginx, strict RDS security group scoping, and verified certificate authority bundles (global-bundle.pem).
- Production Enhancements: discuss how the architecture could be scaled commercially (e.g., placing the client on AWS CloudFront/S3 directly, utilizing an Application Load Balancer with Auto-Scaling Groups, and implementing Redis caching for place queries).

References

Appendix A: AI Usage Declaration form